Using Spring

Spring defines a half dozen or so subcommands that can all be called with syntax like bundle exec spring <whatever>. Let's try one:

```
$ bundle exec spring rspec
Running via Spring preloader in process 60422

Randomized with seed 36292
.....................................................................

Finished in 5.07 seconds (files took 0.66205 seconds to load)
72 examples, 0 failures
```

After I run the tests, I can check the status of Spring again:

```
$ spring status
Spring is running:

60412 spring server | gatherer | started 33 secs ago
60413 spring app    | gatherer | started 33 secs ago | test mode
```

After I run my tests, Spring stays running in the background, as confirmed by the spring status update. If I then enter the Rails console or Rails server, Spring will start up another instance in the development environment and hold onto both of them in the background.

If I then run my tests again with another spring rspec, the tests will start noticeably faster—but sadly, not in a way I can show you by running another set of commands. Since the part that is sped up is the part before Ruby starts timing the tests, the elapsed time shown in the terminal won't change. Trust me, though: depending on your setup you'll see a difference of 5 to 10 seconds. Given that the tests take less than 10 seconds to run, that's a significant change in how fast you can get feedback, especially if you just run a subset of fast or failing tests. It's the difference between being able to run tests without losing focus and not being able to do so.

Spring is a little bit more aggressive about reloading than Rails is in development mode; for instance, it restarts itself when a file changes in the config/initializer directory. However, you'll need to restart Spring if you add files or update gems, and I'm normally prepared to restart Spring any time things look the slightest bit strange.

You don't actually need to restart Spring; you can stop it with the command spring stop. Spring will automatically start back up the next time it's invoked.